

Bono 1: Permutaciones y k-permutaciones

Diseñe un programa que reciba dos enteros n y r , con $0 \leq r \leq n$, y calcule el número de formas de ordenar r objetos distintos tomados de un conjunto de n objetos distintos.

El programa debe calcular:

$${}_nP_r = \frac{n!}{(n-r)!}$$

El programa debe permitir:

1. Calcular $n!$
2. Calcular ${}_nP_k$
3. Validar que n y r sean enteros no negativos
4. Mostrar el procedimiento usado
5. Comparar al menos dos casos, por ejemplo $10P3$ y $20P5$

Extensión opcional: comparar una implementación recursiva y una iterativa del factorial.

Descripción matemática

Las permutaciones son una técnica de conteo para averiguar todas las formas en las que se puede reordenar un conjunto de n elementos. La fórmula para lograr esto es:

$$P(n) = n!$$

Esto solo aplica cuando todos los elementos del conjunto son indistinguibles y en fila. Si están en círculo usamos:

$$P_c(n) = (n-1)!$$

Y para elementos repetidos usamos combinación con repetición:

$$P_r(n) = \frac{n!}{n_1! n_2! \dots n_k!}$$

Donde $n_1, n_2, \dots$, son las repeticiones de cada elemento.

Por el otro lado, la k-permutación es un tipo de permutación en el que se seleccionan k elementos de un conjunto de n elementos y cuenta cuantas posibilidades de elegir hay. La fórmula para esto es:

$${}_nP_k = P(n, k) = \frac{n!}{(n-k)!}$$

Es necesario que $k \leq n$, porque si no, $n - k$ es negativo, y por ende $(n - k)!$ no existe.

En la permutación el orden de selección de los elementos siempre importa. La permutación puede considerarse un caso especial de la k-permutación donde $k = n$.

Implementación

Permutaciones

En la implementación más básica, la librería math de Python ofrece las funciones `math.factorial` y `math.perm` que podemos usar directamente:

```
import math as m

print(m.factorial(7)) # -> 5040

# math.perm ofrece ambas formas de permutación de antemano
print(m.perm(7)) # -> 5040
print(m.perm(7, 4)) # -> 840
```

Sin embargo vamos a crear dos alternativas a la permutación, una iterativa y una recursiva. Ambas bien conocidas en programación. Usaremos anotaciones de tipo para mejor legibilidad y errores para manejo de forma externa.

```
def iterative_factorial(n: int) -> int:
    """Calcula el factorial de un entero dado usando iteración"""

    # Error si el valor no es entero
    if type(n) is not int:
        raise TypeError(f"Valor no válido: {n}")

    # Error de valor si es menor que cero
    if n < 0:
        raise ValueError(f"Valor no válido: {n}")

    fact: int = 1
    for i in range(n):
        fact *= i + 1

    return fact

    # También podríamos usar esta opción (mueve el import al inicio si quieres)
    # import functools as f
    # return f.reduce(lambda x, y: x * y, range(1, n + 1))

def recursive_factorial(n: int) -> int:
    """Calcula el factorial de un entero dado usando recursión"""

    # Error si el valor no es entero
    if type(n) is not int:
        raise TypeError(f"Valor no válido: {n}")

    # Error de valor si es menor que cero
    if n < 0:
        raise ValueError(f"Valor no válido: {n}")

    # Caso base y recursivo (operador ternario)
    return n * recursive_factorial(n - 1) if n else 1
```

Este último aprovecha una propiedad de los factoriales:

$$n! = n(n-1)!$$

Si se repite este proceso con el factorial resultante de la derecha, podemos «desenrollar» un factorial hasta llegar a la forma base $0! = 1$. La forma recursiva desenrolla hasta llegar al caso base y procede luego con los productos. Por ejemplo:

$$\begin{aligned} 7! &= 7 \cdot 6! \\ 7! &= 7 \cdot 6 \cdot 5! \\ 7! &= 7 \cdot 6 \cdot 5 \cdot 4! \\ 7! &= 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3! \\ &\dots \\ 7! &= 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot 1 \\ 7! &= 5040 \end{aligned}$$

La forma iterativa simplemente hace:

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

Rendimiento de la versión recursiva

Ambos algoritmos (iterativo y recursivo) hacen el mismo proceso (multiplicar los números desde 1 hasta n) pero el método recursivo es menos eficiente por el consumo de memoria y stack (pila de llamadas).

A menos de que el código se someta a optimizaciones como la [optimización de llamada de cola \(TCO\)](#) a nivel de compilador o esté escrito en lenguajes funcionales puros como Haskell, el método recursivo posee el riesgo de crear un desborde de pila de llamadas (stack overflow) o un RecursionError (en Python) y consumir tanta memoria como lo diga el número a calcular.

Para el caso específico de Python, este lenguaje no es funcional puro ni ofrece optimizaciones de cola en el intérprete, por lo que se recomienda usar más el enfoque iterativo. O también el TCO manual, aunque es un poco... menos elegante:

```
def recursive_factorial_tco(n: int, acum: int = 1) -> int:
    """Implementa el factorial de un número dado usando recursión y TCO"""

    # Error de tipo si el valor no es entero
    if type(n) is not int:
        raise TypeError(f"Valor no válido: {n}")

    # Error de valor si es menor que cero
    if n < 0:
        raise ValueError(f"Valor no válido: {n}")

    # Caso base
    if n == 0:
        return acum

    # Optimización de cola
    return recursive_factorial_tco(n - 1, acum * n)
```

Este proceso funciona igual que la recursión, pero en lugar de desenrollar y luego hacer el producto, desenrolla un paso y luego multiplica el término nuevo, por ejemplo:

$$\begin{aligned} 7! &= 7 \cdot 6! \\ 7! &= 42 \cdot 5! \\ 7! &= 210 \cdot 4! \\ 7! &= 840 \cdot 3! \\ &\dots \\ 7! &= 5040 \end{aligned}$$

Sin embargo esto no nos salva del `RecursionError` de Python.

K-permutaciones

Para las k-permutaciones podríamos usar los factoriales anteriores, o `math.perm`. Sin embargo vamos a indagar de que van las k-permutaciones para hallar una optimización. La fórmula de las k-permutaciones es.

$$nPk = \frac{n!}{(n-k)!}$$

Si desenrollamos el factorial $n!$ hasta llegar a $(n-k)!$ obtenemos:

$$\begin{aligned} nPk &= \frac{n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-k+1) \cdot \cancel{(n-k)!}}{\cancel{(n-k)!}} \\ nPk &= n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-k+1) \end{aligned}$$

Esta es de hecho la forma original de la k-permutación, que es una aplicación del principio del producto (o de las cajas) para la elección sucesiva de elementos siguiendo los criterios de la permutación. Con esta fórmula podemos obtener la k-permutación usando un único bucle de forma más eficiente:

```
def k_permutation(n: int, k: int) -> int:
    """Implementa la k-permutación de un conjunto de n elementos para elegir k
    elementos"""

    # Error de tipo si los valores no son enteros
    if type(n) is not int or type(k) is not int:
        raise TypeError(f"Valores no válidos: {n}, {k}")

    # Error si alguno es menor que cero o si n es menor que k
    if k < 0 or n < 0 or n < k:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    result: int = 1
    for i in range(n - k + 1, n + 1):
        result *= i

    return result
```

Aplicación de consola

Para una aplicación de consola solo usaremos `prints` e `inputs` para seguir un flujo de usuario básico. También se incluyen verificaciones de errores bien manejados y un código limpio y modular.

El programa estará en el archivo `bono_1_permutaciones/main.py`

Pruebas y casos especiales

Para probar este sistema vamos a probar cuatro casos, con varios ejemplos diferentes para cada caso y por cada algoritmo:

1. Números correctos en un rango común (1-100)
2. Uso de negativos y del cero
3. Números grandes (superiores a 1000)
4. Entradas incorrectas (mal rango o tipo incorrecto)

Este programa estará en el archivo `bono_1_permutaciones/tests.py`

Comentarios y extras

- En algún momento hice un prototipo en una libreta de Jupyter, pero rápidamente se volvía lento y con bugs típicos de las libretas en VSCode. Decidí borrarla.
- También pensé en usar Julia en lugar de Python, pero no lo conozco lo suficientemente bien para usarlo de forma cómoda. Solo como curiosidad, así sería el factorial recursivo allí.

```
function recursive_factorial(n::Int64)::Int64
    if n < 0
        throw(DomainError("Valor no válido: $(n)"))
    end
    return n != 0 ? n * recursive_factorial(n - 1) : 1
end
```

- Uso Python 3.13.5 para este ejercicio. Pero funcionará en cualquier Python moderno.
- Esta documentación está hecha con Typst. Revisa en la carpeta docs/source/ para ver los archivos fuente. Para compilar cualquier cambio [instala typst](#) en tu dispositivo y compila con:

```
typst watch docs/source/archivo.typ [docs/archivo.pdf]
```

Los archivos fuente se ubican en docs/source y los compilados en docs, ambos con el mismo nombre. Con watch, Typst compila en cada cambio que guardes mientras mantengas abierta la terminal. Si omites la segunda ruta, se compila en PDF al lado del archivo fuente.

- Respecto al uso de IA, se usó para identificar errores más rápido y hacer consultas varias. El contenido, código y documentación están íntegramente hechos por un humano.

Referencias

- Murat Asian (2023). *Tail Call Optimization*. Medium. <https://medium.com/@murataslan1/tail-call-optimization-45321f6fe863>
- Datacamp (2025). *Recursión en Python: Conceptos, ejemplos y consejos* <https://www.datacamp.com/es/tutorial/recursion-in-python>